Linux Advanced Practice

1. Difference between `sh` vs `bash`
2. Difference Between `[[` vs `[` vs `(` vs `((` in Bash
3. Sorting FASTA Sequences by Length in Linux
4. Question 1. Histogram of Sequence Identities of Inter- and Intra-group
5. Question 2. Sequence Identity Matrix of Inter- and Intra-groups

1. Difference between `sh` vs `bash`

`sh` and `bash` are both command-line interpreters in Unix-like systems, but there are some key differences:

1. Basic Functionality:

- **`sh` (Shell)**: Originally the standard shell in UNIX systems, referring to the **Bourne Shell**. It provides basic command-line interface and scripting features such as variables, conditionals, loops, etc.
- **`bash` (Bourne Again Shell)**: A shell developed by the GNU project. It's compatible with `sh` but extends functionality with features like improved scripting capabilities, better error handling, and more interactive features.

2. Feature Differences:

- `bash` includes more modern features, such as command auto-completion, command history, aliases, and advanced scripting features like arrays and arithmetic expansion, which `sh` lacks.
- `bash` supports more built-in commands and syntax extensions (e.g., arrays, arithmetic expressions), while `sh` is more basic and only supports fundamental commands.=

3. Compatibility:

- `bash` is compatible with `sh`, meaning most `sh` scripts will work in `bash`.
- However, since `bash` includes features that `sh` does not, some scripts written for `bash` may not run in `sh`.

4. Default Environment:

- On many Linux systems, `/bin/sh` is a symbolic link to `bash` for compatibility.
- On some systems (e.g., Debian and Ubuntu), `/bin/sh` links to `dash`, which is a lightweight, POSIX-compliant shell.

In summary:

`bash` is an enhanced version of `sh`, offering more features and extensions. However, if a script needs to run across various Unix systems, it's often better to use `sh` to ensure compatibility.

Fixing Shell History Navigation and Backspace Issues in Linux

Issue 1: Arrow Keys Showing `^[[A` `^[[B` Instead of Navigating History

Cause:

This issue usually occurs when the shell is not interpreting key sequences correctly, often due to:

- The wrong shell being used (`sh` instead of `bash`).
- An incorrect or missing terminal configuration.

Solution:

Ensure you are using Bash and run the following command:

```
echo $SHELL
chsh -s /bin/bash
```

2. Difference Between `[[` vs `[` vs `(` vs `((` in Bash

The operators `[[`, `[`, `(`, and `((` in Bash are used for different purposes, especially in conditional expressions, arithmetic operations, and control structures. Here's a breakdown of each:

1. `[` Single Bracket

- **Purpose**: The single bracket is used for **basic test** commands, primarily to evaluate conditions like file tests, string comparisons, or checking if a variable is set.
- **Syntax**: `[ expression ]`
- **Examples**:
  - File test: `[ -e file.txt ]` checks if `file.txt` exists.
  - String comparison: `[ "$a" = "$b" ]` checks if `$a` is equal to `$b`.
  - Numeric comparison: `[ $a -eq $b ]` checks if `$a` is equal to `$b`.
- **Limitations**:
  - It doesn't support advanced features like regex matching, multiple conditions, or direct arithmetic operations.
  - It requires spaces around the brackets.

2. `[[` Double Bracket

- **Purpose**: The double bracket is an **extended test command** that supports more advanced features, such as pattern matching, logical operators, and is generally more flexible than `[`.
- **Syntax**: `[[ expression ]]`
- **Examples**:
  - String comparison with pattern matching: `[[ "$a" == z* ]]` checks if `$a` starts with "z".
  - Logical AND: `[[ $a -eq 5 && $b -eq 10 ]]` checks if both conditions are true.
  - Regular expression matching: `[[ "$a" =~ ^[0-9]+$ ]]` checks if `$a` is a number.
- **Advantages**:
  - Supports logical operators like `&&` (AND), `||` (OR), and negation `!`.
  - Works with regular expressions.
  - Handles spaces and special characters better (e.g., no need to quote variables).

3. `(` Parentheses

- **Purpose**: Parentheses are used for **subshell execution**. Anything inside parentheses runs in a new subshell, which means it has its own environment and any changes to variables or working directories won't affect the parent shell.
- **Syntax**: `( command )`
- **Example**:
  - `(cd /tmp; ls)` will change the directory to `/tmp` and list its contents in a subshell. Afterward, the current directory in the parent shell remains unchanged.
- **Usage**:
  - Used for running commands in a subshell.
  - Changes made in the subshell do not affect the parent shell (e.g., variable assignments, working directory changes).

4. `((` Double Parentheses

- **Purpose**: The double parentheses are used for **arithmetic operations** and **evaluating mathematical expressions**.
- **Syntax**: `(( expression ))`
- **Examples**:
  - Arithmetic operations: `(( a = b + 1 ))` performs arithmetic and assigns the result to `a`.
  - Comparison: `(( a > b ))` evaluates whether `a` is greater than `b`.
  - Increment: `(( a++ ))` increments `a` by 1.
  - `(( ))` also supports logical operators (e.g., `&&`, `||`).
- **Advantages**:
  - You don't need to use `$` to refer to variables inside `(( ))` (e.g., `((a = a + 1))` instead of `((a = $a + 1))`).
  - Supports more complex arithmetic, including bitwise operators.

Summary of Differences:

Operator	Purpose	Syntax	Supports
`[`	Basic test (file, string, and numeric checks)	`[ expression ]`	Basic conditions, file tests
`[[`	Extended test (pattern matching, regex, etc.)	`[[ expression ]]`	Logical operators, regex, pattern matching
`(`	Subshell execution	`( command )`	Runs commands in a subshell
`((`	Arithmetic operations	`(( expression ))`	Arithmetic, numeric comparisons, increment/decrement

Each operator has its specific use case, and understanding their differences helps you choose the correct one for your script.

3. Sorting FASTA Sequences by Length in Linux

1. Convert Multi-line Sequences to Single-line

```
>2
ATCG
ATCG
ATCG
>1
ATCG
>3
ATCGATCGATCGATCG
```

Use `awk` to merge multi-line sequences into a single line while keeping headers:

```
awk '/^>/ {if (seq) print seq; print; seq=""; next} {seq=seq$0} END {if (seq) print
seq}' a.fa > single_line.fa
```

2. Compute Sequence Length and Sort

Use awk to calculate sequence lengths and sort to order them numerically:

```
%awk '/^>/ {if (seq) print length(seq), header, seq; header=$0; seq=""; next}
{seq=seq$0} END {if (seq) print length(seq), header, seq}' single_line.fa | sort -n >
sorted_sequences.txt
% cat sorted_sequences.txt
4 >1 ATCG
12 >2 ATCGATCGATCG
16 >3 ATCGATCGATCGATCG
```

3. Restore Standard FASTA Format

Extract only headers and sequences:

```
%awk '{print $2"\n"$3}' sorted_sequences.txt > sorted_sequences.fasta
%cat sorted_sequences.fasta
>1
ATCG
>2
ATCGATCGATCG
>3
ATCGATCGATCGATCG
```

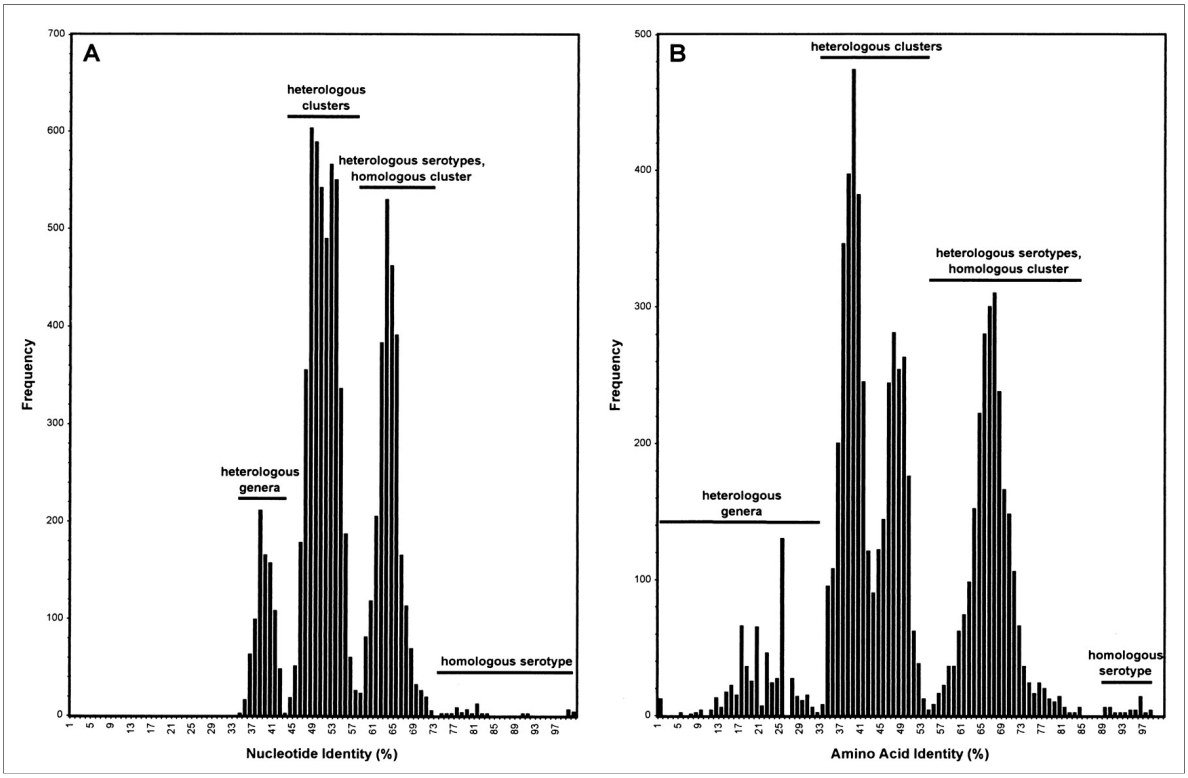
(Optional) Using biopython

```
In [ ]: import argparse
        from Bio import SeqIO

def sort_fasta_by_length(input_file, output_file, reverse=False):
    # Read sequences from the input file
    records = list(SeqIO.parse(input_file, "fasta"))
    # Sort sequences by length
    records.sort(key=lambda r: len(r.seq), reverse=reverse)
    # Write sorted sequences to the output file
    SeqIO.write(records, output_file, "fasta")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Sort FASTA sequences by length.")
    parser.add_argument("-i", "--input", required=True, help="Input FASTA file.")
    parser.add_argument("-o", "--output", required=True, help="Output FASTA file.")
    parser.add_argument("-r", "--reverse", action="store_true", help="Sort in descending order.")
    args = parser.parse_args()
    sort_fasta_by_length(args.input, args.output, args.reverse)
```

Question 1. Histogram of Sequence Identities of Inter- and Intra-group

- Given multiple sets of nucleotide or protein sequences in FASTA format, calculate both **intra-group** and **inter-group** sequence identity, then visualize the results using a histogram.

- Expected Result: Fig. 3. Frequency distribution of pairwise identity scores for comparison of VP1 nucleotide (A) and deduced amino acid (B) sequences. from DOI: 10.1128/jvi.73.3.1941-1948.1999



Question 2. Sequence Identity Matrix of Inter- and Intra-groups

- Given multiple sets of nucleotide or protein sequences in FASTA format, calculate both **intra-group** and **inter-group** sequence identity, then visualize the results using a histogram.

- Expected Result:

Group	Group 1	Group 2	Group 3
Group 1	95-100	85-90	82-88
Group 2	-	96-100	82-88
Group 3	-	-	99-100